

CS 170 Proficiency Quiz 2

Write a program that **constructs the objects and calls the methods** specified as comments in your starter file. You will use the **class documented below**.

Carefully read the class documentation; while it is similar to the classes you have used in Chapter 2, ***it is not the same***. You will have 30 minutes to complete the problem. You may not access any notes or files, other than the Java quick reference I've supplied, or communicate with anyone in any way. When you have finished and submitted the quiz, check with me to be sure I have received it.

You can check your problem by using **Tools->Compile** to see if your code is syntactically correct. To test your code, use **Tools->Go**. To see what the output looks like (without testing it), use **Tools->Run in Console**.

Creating Objects and Calling Methods

Inside the class **PQ02.java**, create the requested objects and call the requested methods as specified in the comments.

- You'll find the documentation for the class starting on the next page.
- This problem will be graded 95% on correctness and 5% on style.
- You may check your quiz as often as you like before you submit it.
- The last time you check your quiz is the one that is used.
- If you don't check your quiz, you will get a 0 on the quiz.
- If time runs out, your quiz will be automatically submitted.

Class Color

java.lang.Object
Color

```
public class Color
extends java.lang.Object
```

A Color class for CS 170 - Fall 2018

Version:

CS170-F18

Author:

Stephen Gilbert

Constructor Summary

Constructors**Constructor and Description**

`Color()`

Creates a black Color object.

`Color(Color color)`

Makes a duplicate of the given color.

`Color(double redPercent, double greenPercent, double bluePercent)`

Creates an opaque RGB color.

`Color(int red, int green, int blue)`

Creates an opaque RGB color.

`Color(int red, int green, int blue, int alpha)`

Creates an RGBA color with an alpha or transparency channel.

Method Summary

All Methods**Instance Methods****Concrete Methods****Modifier and Type****Method and Description**

void

`brighter()`

Changes this color to a brighter version.

void

`darker()`

Changes this color to a brighter version.

int

`getBlue()`

The amount of blue in this color.

int

`getGreen()`

The amount of green in this color.

int

`getOpacity()`

The opacity value of this color.

int

`getRed()`

The amount of red in this color.

java.lang.String

`toString()`

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

Color

```
public Color()
```

Creates a black Color object.

Color

```
public Color(Color color)
```

Makes a duplicate of the given color.

Parameters:

color - the color to copy.

Color

```
public Color(double redPercent,  
             double greenPercent,  
             double bluePercent)
```

Creates an opaque RGB color.

Parameters:

redPercent - the percentage of red (0.0 to 1.0)

greenPercent - the percentage of green (0.0 to 1.0)

bluePercent - the percentage of blue (0.0 to 1.0)

Color

```
public Color(int red,
             int green,
             int blue)
```

Creates an opaque RGB color.

Parameters:

red - the amount of red (0..255)

green - the amount of green (0..255)

blue - the amount of blue (0..255)

Color

```
public Color(int red,
             int green,
             int blue,
             int alpha)
```

Creates an RGBA color with an alpha or transparency channel.

Parameters:

red - the amount of red (0..255)

green - the amount of green (0..255)

blue - the amount of blue (0..255)

alpha - the amount of transparency (0=transparent...255=opaque)

Method Detail

brighter

```
public void brighter()
```

Changes this color to a brighter version.

darker

```
public void darker()
```

Changes this color to a brighter version.

getBlue

```
public int getBlue()
```

The amount of blue in this color.

Returns:

the amount of blue in this color.

getGreen

```
public int getGreen()
```

The amount of green in this color.

Returns:

the amount of green in this color.

getOpacity

```
public int getOpacity()
```

The opacity value of this color. 255 means the color is opaque, while 0, is fully transparent.

Returns:

the amount of opacity in this color.

getRed

```
public int getRed()
```

The amount of red in this color.

Returns:

the amount of red in this color.

toString

```
public java.lang.String toString()
```

Overrides:

toString in class java.lang.Object

PACKAGE

CLASS

TREE

INDEX

HELP

PREV CLASS

NEXT CLASS

FRAMES

NO FRAMES

ALL CLASSES

SUMMARY: NESTED | FIELD | CONSTR | METHOD

DETAIL: FIELD | CONSTR | METHOD

Variable and Constant Declarations

Type **Name** **Initial value**
/
/
/
int cansPerPack = 6;

final double CAN_VOLUME = 0.335;

Method Declaration

Modifiers **Return type** **Parameter type and name**
/
/
/
public static double cubeVolume(double sideLength)
{
 double volume = sideLength * sideLength * sideLength;
 return volume;
}

Exits method and returns result.

Conditional Statement

Condition
/
if (floor >= 13)
{
 actualFloor = floor - 1;
}
else if (floor >= 0) Second condition (optional)
{
 actualFloor = floor;
}
else
{
 System.out.println("Floor negative");
}
}

Executed when condition is true

Executed when all conditions are false (optional)

Loop Statements

Condition
/
while (balance < TARGET)
{
 year++;
 balance = balance * (1 + rate / 100);
}

Executed while condition is true

Initialization **Condition** **Update**
/
/
/
for (int i = 0; i < 10; i++)
{
 System.out.println(i);
}

do
{
 System.out.print("Enter a positive integer: ");
 input = in.nextInt();
}
while (input <= 0);

Loop body executed at least once

Set to a new element in each iteration

An array or collection
/
for (double value : values)
{
 sum = sum + value;
}

Executed for each element

String Operations

```
String s = "Hello";  
int n = s.length(); // 5  
char ch = s.charAt(1); // 'e'  
String t = s.substring(1, 4); // "ell"  
String u = s.toUpperCase(); // "HELLO"  
if (u.equals("HELLO")) ... // Use equals, not ==  
for (int i = 0; i < s.length(); i++)  
{  
    char ch = s.charAt(i);  
    Process ch  
}
```

Mathematical Operations

Math.pow(x, y)	Raising to a power x^y
Math.sqrt(x)	Square root $\sqrt{x}$
Math.log10(x)	Decimal log $\log_{10}(x)$
Math.abs(x)	Absolute value $ x $
Math.sin(x)	} Sine, cosine, tangent of x (x in radians)
Math.cos(x)	
Math.tan(x)	

Class Declaration

```
public class CashRegister  
{  
    private int itemCount;  
    private double totalPrice;  
}  
  
public void addItem(double price)  
{  
    itemCount++;  
    totalPrice = totalPrice + price;  
}  
...  
}
```

Instance variables

Method

Selected Operators and Their Precedence

(See Appendix B for the complete list.)

[]	Array element access
++ -- !	Increment, decrement, Boolean <i>not</i>
* / %	Multiplication, division, remainder
+ -	Addition, subtraction
< <= > >=	Comparisons
== !=	Equal, not equal
&&	Boolean <i>and</i>
	Boolean <i>or</i>
=	Assignment

Input

```
Scanner in = new Scanner(System.in);
// Can also use new Scanner(new File("input.txt"));

int n = in.nextInt();
double x = in.nextDouble();
String word = in.next();
String line = in.nextLine();

while (in.hasNextDouble())
{
    double x = in.nextDouble();
    Process x
}
```

Output

```
System.out.print("Enter a value: ");

System.out.println("Volume: " + volume);

System.out.printf("%-10s %10d %10.2f", name, qty, price);

PrintWriter out = new PrintWriter("output.txt");

out.close();
```

Does not advance to new line.

Use + to concatenate values.

Field width **Precision**

Left-justified string **Integer** **Floating-point number**

Use print/println/printf to write output to file.

Remember to close output file.

Arrays

```
int[] numbers = new int[5];
int[] squares = { 0, 1, 4, 9, 16 };
int[][] magicSquare =
{
    { 16, 3, 2, 13},
    { 5, 10, 11, 8},
    { 9, 6, 7, 12},
    { 4, 15, 14, 1}
};

for (int i = 0; i < numbers.length; i++)
{
    numbers[i] = i * i;
}

for (int element : numbers)
{
    Process element
}

System.out.println(Arrays.toString(numbers));
// Prints [0, 1, 4, 9, 16]
```

Element type **Element type** **Length**

All elements are zero.

Array Lists

```
ArrayList<String> names = new ArrayList<String>();

names.add("Ann");
names.add("Cindy"); // [Ann, Cindy], names.size() is now 2

names.add(1, "Bob"); // [Ann, Bob, Cindy]
names.remove(2); // [Ann, Bob]
names.set(1, "Bill"); // [Ann, Bill]

String name = names.get(0); // Gets "Ann"
System.out.println(names); // Prints [Ann, Bill]
```

Initially empty

Use wrapper type, Integer, Double, etc., for primitive types.

Element type

Add elements to the end

Linked Lists, Sets, and Iterators

```
LinkedList<String> names = new LinkedList<String>();
names.add("Bob"); // Adds at end

ListIterator<String> iter = names.listIterator();
iter.add("Ann"); // Adds before current position

String name = iter.next(); // Returns "Ann"
iter.remove(); // Removes "Ann"

Set<String> names = new HashSet<String>();
names.add("Ann"); // Adds to set if not present
names.remove("Bob"); // Removes if present

Iterator<String> iter = names.iterator();
while (iter.hasNext())
{
    Process iter.next()
}
```

Maps

```
Map<String, Integer> scores = new HashMap<String, Integer>();

scores.put("Bob", 10);
Integer score = scores.get("Bob");

for (String key : scores.keySet())
{
    Process key and scores.get(key)
}
```

Key type **Value type**

Returns null if key not present